

Boost.Int128: A C++14 Library for Portable and Performant 128-Bit Integer Arithmetic

Matt Borland ¹

¹ The C++ Alliance, United States

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 21 August 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Boost.Int128 is a header-only, C++14 implementation of 128-bit integer arithmetic through two types, `int128` and `uint128`, that behave like built-in integers and integrate with the C++ Standard Library and other Boost libraries (Boost Authors, 2026). Both types are exactly 128 bits wide on every supported platform, and both are usable unchanged in GPU kernels through CUDA (Nickolls et al., 2008) and SYCL (Reyes et al., 2020). The library is available at <https://github.com/boostorg/int128>, and in the Boost distribution starting with version 1.93.

Statement of need

C++ specifies no integer type wider than `long long`, which in practice means 64 bits, yet quantities exceeding that range are common. An IPv6 address (Hinden & Deering, 2006) and a UUID (Davis et al., 2024) are each 128 bits, so each needs a 128-bit integer or a byte array that cannot be compared, masked, or incremented directly. The widening 64x64 to 128-bit product is the central primitive of unbiased random integer generation in an interval (Lemire, 2019) and of shortest round-trip floating-point formatting (Adams, 2018). GCC and Clang expose `__int128` only on 64-bit targets, with no `std::numeric_limits`, `<iostream>`, or `std::hash` support, and MSVC lacks it entirely, so dependent code either restricts its portability or maintains a second implementation. The portable alternative, a general multiprecision integer, pays for that generality in object size and in throughput. The types began as the arithmetic backend of Boost.Decimal (Borland & Kormanyos, 2026), which needs an exact 128-bit product on every platform Boost supports, and were extracted and generalized once that need proved broader than one library. What is needed is a type that is exactly 16 bytes everywhere, as fast as the compiler's extension where that exists, and usable across the host-to-device boundary without an interchange format.

State of the field

The portable host implementations are Abseil's `absl::int128` and `absl::uint128` (Google, 2017), Boost.Multiprecision's 128-bit `cpp_int` instantiation (Kormanyos & Maddock, 2026), and MSVC's undocumented `std::_Signed128` and `std::_Unsigned128` (Microsoft Corporation, 2026). Abseil is a close functional match, but it requires C++17 and GCC 10 or later and carries no device annotations. Boost.Multiprecision's types instantiate a general number template rather than a purpose-built class, so they carry an extra word of bookkeeping and cost more per operation.

On the device side, `nvcc` accepts `__int128` only when the host compiler supports it (NVIDIA Corporation, 2026), which excludes every MSVC and 32-bit host, and SYCL's `spir64` target has no native 128-bit integer at all (Khronos SYCL Working Group, 2023). The implementations

40 filling this gap, such as CUDA-uint128 (Seizert, 2016), are unsigned only and tied to one
41 toolchain. We are not aware of another implementation offering one 128-bit type usable on
42 the host and in both CUDA and SYCL device code with bitwise identical results.

43 Software design

44 The library requires only C++14, is header-only, and has no external dependencies, so a
45 user need only `#include <boost/int128.hpp>`, even with a toolchain as old as g++-5. A
46 single-header amalgamation is published, and under C++20 the library can be consumed
47 as a module via `import boost.int128`. Relaxed C++14 constexpr covers nearly the whole
48 interface.

49 Each type is a struct of two `std::uint64_t` words ordered by target endianness, so the object
50 representation and alignment match a native 128-bit integer. Keeping both words unsigned
51 lets the compiler treat the pair as one wide access, so loops over these types vectorize rather
52 than scalarize. Operators forward to a native 128-bit type or intrinsic where one exists,
53 and follow Knuth (Knuth, 1998) and Warren (Warren, 2013) over the two words elsewhere,
54 with identical results. The library also supplies the interfaces the extension lacks, among
55 them `<limits>`, `<bit>`, `<charconv>`, `<format>`, `<iostream>`, hashing, `Boost.Random` and
56 `Boost.Math` integration, saturating arithmetic, the C23 checked-arithmetic interface (ISO/IEC,
57 2024), and the `div_*` family of P3724 (Schultke, 2025).

58 Continuous integration runs the test suite natively on Linux, macOS, and Windows across
59 `x86_64`, `x86_32`, `aarch64`, `ARM32v7`, `ARM64`, and big-endian `s390x`, plus `PPC64LE` under
60 QEMU, using GCC 5 and later, Clang 5 and later, Visual Studio 2017 and later, Intel oneAPI
61 DPC++, and NVCC. Because half of these targets have no native 128-bit type, the software
62 fallbacks are tested as thoroughly as the intrinsic paths. GPU support is expressed through
63 one annotation macro, `BOOST_INT128_HOST_DEVICE`, which expands to `__host__ __device__`
64 under `nvcc` and `SYCL_EXTERNAL` under a SYCL compiler. More than eighty CUDA and eighty
65 SYCL tests compare device results against the host, so agreement across that boundary is
66 tested rather than assumed.

67 Where a native type exists, `Boost.Int128` matches or beats it: with GCC 14.2 on Linux `x86_64`,
68 `uint128` division and modulo are roughly 16 percent faster than unsigned `__int128`, and
69 signed multiplication is 1.5 times faster on big-endian `s390x`. Where no native type exists, the
70 margins are larger: signed multiplication is 1.7 times faster than `std::Signed128` on MSVC
71 `x86_64` and 6.2 times faster on MSVC `ARM64`, and 32-bit `x86` addition is 8 times faster than
72 `Boost.Multiprecision`. A handful of cells favor an alternative, so the complete tables, rerun
73 on every pull request, are published rather than a summary, at https://develop.int128.cpp.al/int128/u128_benchmarks.html and
74 https://develop.int128.cpp.al/int128/i128_benchmarks.html.
75

76 Figure 1 shows the difference between a 64-bit and a 128-bit product.

```
#include <boost/int128.hpp>
#include <cstdint>
#include <iostream>

int main()
{
    using boost::int128::uint128;

    // Two values that each fit in 64 bits, but whose product does not
    constexpr std::uint64_t a {11400714819323198485ULL};
    constexpr std::uint64_t b {14029467366897019727ULL};

    // A 64-bit product silently wraps modulo 2^64
```

```
std::cout << " 64-bit: a * b = " << a * b << std::endl;

// The same multiplication is exact when the result is 128 bits wide
constexpr uint128 exact {uint128{a} * uint128{b}};
std::cout << "128-bit: a * b = " << exact << std::endl;

// Like a built-in integer, the types work in constant expressions
static_assert(exact / uint128{b} == uint128{a}, "Recovers a exactly");
static_assert(sizeof(uint128) == 16, "Exactly 128 bits on every platform");

return 0;
}

64-bit: a * b = 17693923505768731003
128-bit: a * b = 159945956516994065446882290339781513595
```

77 *Figure 1: The 64-bit product wraps modulo 2^{64} while the `uint128` product is exact and*
78 *compile-time checked. The lower block is the program output.*

79 Research Impact Statement

80 Boost.Int128 supports reproducible numerical research wherever 64 bits of integer precision
81 are not enough. Computational number theory, cryptography, and hashing reduce products of
82 64-bit residues and need the full 128-bit intermediate, as do combinatorial enumeration and
83 exact sums of products. Because the same type gives the same results on every architecture,
84 big-endian or little-endian, and with or without a native 128-bit type underneath, a result
85 computed on one machine is bit-for-bit reproducible on another.

86 The device support extends that guarantee to accelerators. A researcher can port a kernel
87 to a GPU without substituting a numeric type and can validate the port against the host
88 because the two agree by specification. Replacing a floating-point accumulator with a 128-bit
89 integer one makes a parallel reduction independent of how the work is partitioned, since integer
90 addition is associative. Supporting both CUDA and SYCL means this holds across vendors.

91 AI usage disclosure

92 Generative AI tools were used in developing this library and preparing this manuscript, for
93 drafting and editing prose, platform-specific optimizations, and generating test cases. All
94 library code, benchmark results, and text were reviewed and verified by the author, who takes
95 full responsibility for their correctness.

96 Acknowledgements

97 We thank The C++ Alliance for sponsoring this work, the domain experts who peer-reviewed
98 the library in July 2026 before its acceptance into the Boost library collection, and Arnaud
99 Bechler for managing that review.

100 References

- 101 Adams, U. (2018). Ryu: Fast float-to-string conversion. *Proceedings of the 39th ACM*
102 *SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*,
103 270–282. <https://doi.org/10.1145/3192366.3192369>

- 104 Boost Authors. (2026). Boost C++ libraries. In *GitHub repository*. GitHub. <https://github.com/boostorg/boost>
- 105
- 106 Borland, M., & Kormanyos, C. (2026). Boost.decimal: A c++14 library for decimal floating
107 point arithmetic. *Journal of Open Source Software*, 11(124), 10345. <https://doi.org/10.21105/joss.10345>
- 108
- 109 Davis, K. R., Peabody, B., & Leach, P. J. (2024). *Universally unique IDentifiers (UUIDs)*
110 (RFC 9562). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9562>
- 111 Google. (2017). Abseil common libraries. In *GitHub repository*. GitHub. <https://github.com/abseil/abseil-cpp>
- 112
- 113 Hinden, R. M., & Deering, S. E. (2006). *IP version 6 addressing architecture* (RFC 4291).
114 Internet Engineering Task Force. <https://doi.org/10.17487/RFC4291>
- 115 ISO/IEC. (2024). *ISO/IEC 9899:2024, Programming Languages - C*. International Organization
116 for Standardization. <https://www.iso.org/standard/82075.html>
- 117 Khronos SYCL Working Group. (2023). *SYCL 2020 Specification*. The Khronos Group.
118 <https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html>
- 119 Knuth, D. E. (1998). *The art of computer programming, volume 2: Seminumerical algorithms*
120 (3rd ed.). Addison-Wesley. ISBN: 9780201896848
- 121 Kormanyos, C., & Maddock, J. (2026). Boost.Multiprecision. In *GitHub repository*. GitHub.
122 <https://github.com/boostorg/multiprecision>
- 123 Lemire, D. (2019). Fast random integer generation in an interval. *ACM Transactions on*
124 *Modeling and Computer Simulation*, 29(1), 1–12. <https://doi.org/10.1145/3230636>
- 125 Microsoft Corporation. (2026). Microsoft C++ Standard Library: __msvc_int128.hpp. In
126 *GitHub repository*. GitHub. https://github.com/microsoft/STL/blob/main/stl/inc/__msvc_int128.hpp
- 127
- 128 Nickolls, J., Buck, I., Garland, M., & Skadron, K. (2008). Scalable parallel programming with
129 CUDA. *Queue*, 6(2), 40–53. <https://doi.org/10.1145/1365490.1365500>
- 130 NVIDIA Corporation. (2026). *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- 131
- 132 Reyes, R., Brown, G., Burns, R., & Wong, M. (2020). SYCL 2020: More than meets
133 the eye. *Proceedings of the International Workshop on OpenCL (IWOCCL '20)*. <https://doi.org/10.1145/3388333.3388649>
- 134
- 135 Schultke, J. (2025). *P3724: Integer division*. ISO/IEC JTC1/SC22/WG21. <https://wg21.link/p3724>
- 136
- 137 Seizert, C. (2016). CUDA-uint128: A 128 bit unsigned integer class for CUDA. In *GitHub*
138 *repository*. GitHub. <https://github.com/curtisizeert/CUDA-uint128>
- 139 Warren, H. S. (2013). *Hacker's delight* (2nd ed.). Addison-Wesley. ISBN: 9780321842688